

# UNA INTERFAZ QUE HABLA CON LA CADENA Y NO MIENTE

Conectar MetaMask, leer el saldo de la FichaUSB, transferir, escuchar eventos y provocar a propósito los cuatro errores que todo principiante ignora.

| MODALIDAD  | DURACIÓN                                  | REQUISITOS                                         | META                             |
|------------|-------------------------------------------|----------------------------------------------------|----------------------------------|
| En parejas | 85 min en clase<br>+ lo que falte en casa | Node 22, repo del curso,<br>MetaMask,<br>navegador | 8 TODO completos<br>+ 6 capturas |

## CÓMO LEER ESTA GUÍA

Está escrita para que la puedan seguir **sin ayuda**. Cada paso dice qué escribir, qué deberían ver y cómo saber que funcionó. Los comandos están en PowerShell (Windows, la sala); cuando cambian en macOS/Linux, se indica. No hay que compilar nada: la interfaz es **un solo archivo HTML** que se edita, se guarda y se recarga en el navegador. Si algo no sale como aquí se describe, vayan a **«Errores frecuentes»** (sección 11). Todo lo que aparece como «salida esperada» se obtuvo de verdad al preparar el laboratorio.

## REGLAS QUE NO ADMITEN EXCEPCIÓN

- Solo red local de Hardhat o Sepolia. **Nunca** dinero real; la billetera es la del curso.
- La frase semilla no se comparte con nadie, tampoco con el docente. **No se firma lo que no se entiende**: lean lo que muestra MetaMask antes de confirmar.
- En este laboratorio **no se importa ninguna clave privada** en MetaMask: los fondos locales llegan a su cuenta con un script.
- Claves y URLs de RPC con clave: solo en el keystore de Hardhat, jamás en el HTML ni en Git.

## 01 Objetivo, resultado de aprendizaje y entrega

LEER PRIMERO

**Objetivo:** construir una interfaz que lea y escriba en un contrato, gestionando la billetera y los estados de una transacción: pendiente, minada, y los cuatro errores (billetera no instalada, red equivocada, rechazo del usuario, transacción revertida).

**RA6** · Integrar una interfaz Web 3.0 con contratos, gestionando wallets, eventos y estados de transacción.

| EVIDENCIA                  | DETALLE                                                                                         | PESO |
|----------------------------|-------------------------------------------------------------------------------------------------|------|
| Repositorio                | Su <code>index.html</code> con los 8 TODO completos, en el repositorio del equipo.              | 25 % |
| Transferencia real         | Captura del estado verde «confirmada» con su hash. En Sepolia, además, el enlace al explorador. | 20 % |
| Cuatro errores + 2 estados | Seis capturas: sin billetera, red equivocada, rechazo, revertida, pendiente, confirmada.        | 35 % |
| Eventos                    | Evidencia de que el saldo cambió sin recargar la página.                                        | 10 % |
| Reflexión                  | Respuestas a las preguntas de la sección 12.                                                    | 10 % |

Política del curso: sin hash de transacción, el laboratorio no se califica. Si solo trabajan en red local, repitan una transferencia en Sepolia desde casa para que la evidencia sea verificable por terceros.

## 02 El mapa y los archivos

```
material/laboratorio-12/
├─ andamiaje/index.html      <- AQUÍ TRABAJAN: la dApp con 8 TODO
├─ guia/guia-laboratorio-12.pdf <- esta guía
└─ README.md

material/laboratorios-evm/    <- el repo Hardhat del curso
├─ contracts/sl10/FichaUSB.sol <- el token de la Sesión 10
├─ scripts/sl12/preparar-local.js <- despliega FichaUSB local y les da fondos
└─ scripts/sl12/comprobar-ficha.js <- punto de control desde la terminal
```

|        |                                            |        |
|--------|--------------------------------------------|--------|
| PASO 1 | comprobar herramientas                     | 5 min  |
| PASO 2 | servir la página por http://localhost      | 5 min  |
| PASO 3 | un token para probar (red local o Sepolia) | 15 min |
| PASO 4 | configurar la dApp y punto de control      | 5 min  |
| PASO 5 | TODO 1 a 5: detectar, conectar, leer       | 20 min |
| PASO 6 | TODO 6 a 8: transferir, errores, eventos   | 20 min |
| PASO 7 | provocar los cuatro errores y capturarlos  | 15 min |

### POR QUÉ SIN REACT NI COMPILACIÓN

El plan del curso menciona una aplicación React. En la sala puede no estar permitido instalar el conjunto de herramientas de React, así que el laboratorio usa el mismo enfoque de la dApp de referencia: **un HTML con ethers v6 cargado desde jsDelivr**. Se ve cada pieza (Provider, Signer, Contract) sin magia. Para el proyecto pueden usar React + wagmi + RainbowKit (lámina A.18): los conceptos y los estados de error son exactamente los mismos.

## 03 Paso 1 · Comprobar las herramientas

5 MIN

```
node -v          # v22.algo
python --version # 3.algo (solo si van a usar el plan B del paso 2)
cd "ruta\a\laboratorios-evm"
npx hardhat --version # 3.16.0 (si falla: npm install, una sola vez)
```

**MetaMask:** la extensión de la billetera del curso, instalada en Chrome o Firefox y desbloqueada. Copien su dirección (clic en el nombre de la cuenta): la van a necesitar en el paso 3. **La dirección no es secreta**; la frase semilla sí.

### CÓMO SÉ QUE ESTOY LISTO

Las tres versiones responden y tienen su dirección `0x...` copiada en un archivo de notas.

## 04 Paso 2 · Servir la página (y por qué el doble clic no sirve)

5 MIN

Si abren `index.html` con doble clic, el navegador lo carga como `file:///C:/.../index.html`. Con MetaMask eso falla, por dos razones:

- **Chrome no deja actuar a las extensiones en páginas `file://`** salvo que el usuario active, extensión por extensión, «Permitir acceso a URL de archivo». Viene apagado. Sin eso, MetaMask no inyecta `window.ethereum` y la dApp dice «No se detecta una billetera».
- **MetaMask da permisos por sitio** (por origen: esquema + dominio + puerto). Una página `file://` no tiene un origen normal, así que la conexión es poco fiable aunque se active el permiso.

### NO ACTIVEN «PERMITIR ACCESO A URL DE ARCHIVO»

Le daría a la extensión acceso a archivos de su disco. La práctica correcta es la misma que en producción: servir la página por HTTP. En local, por `http://localhost`.

**Opción A · con Node** (la primera vez `npx` pregunta si descarga el paquete `serve`: respondan `y`):

```
cd "ruta\a\material\laboratorio-12\andamiaje"
npx serve .
```

```
Serving!
- Local:   http://localhost:3000
```

**Opción B · con Python** (si la sala no deja descargar `serve`):

```
cd "ruta\a\material\laboratorio-12\andamiaje"
python -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

Abran `http://localhost:3000` (opción A) o `http://localhost:8000` (opción B). Dejen esa terminal abierta: si la cierran, la página deja de cargar.

### PUNTO DE CONTROL 1

La página muestra «FichaUSB · Laboratorio 12 (versión de trabajo)». Al pulsar «Conectar billetera», la caja «Estado» dice en rojo «*Falta completar el TODO 2 (conectar) en el código de index.html.*» Eso es correcto: la página funciona y les avisa qué falta. Si en cambio no pasa nada, abran la consola del navegador (F12 → Console) y busquen el error.

## 05 Paso 3 · Un token para probar

15 MIN · ELIJAN UNA OPCIÓN

| OPCIÓN        | CUÁNDO                                                            | VENTAJA / DESVENTAJA                                                       |
|---------------|-------------------------------------------------------------------|----------------------------------------------------------------------------|
| A · red local | En la sala. No depende de faucets ni de la red de la universidad. | Todo se mina al instante: los estados «pendiente» apenas se ven.           |
| B · Sepolia   | En casa, con la FichaUSB que desplegaron en la Sesión 10.         | Evidencia verificable por cualquiera; hay que tener ETH y FUSB de Sepolia. |

### Opción A · red local de Hardhat

3A.1 · Terminal 1: levanten el nodo local y déjenlo corriendo todo el laboratorio.

```
cd "ruta\a\laboratorios-evm"
npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/
Accounts
=====
WARNING: Funds sent on live network to accounts with publicly known private keys WILL BE LOST.
Account #0: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266 (10000 ETH)
...
```

Esas cuentas y claves son públicas (las conoce todo el mundo): **no las importen** en MetaMask. No hace falta.

3A.2 · MetaMask: agregar la red local. En MetaMask: selector de redes → *Agregar una red personalizada* (el texto exacto cambia entre versiones). Llenen:

| CAMPO                   | VALOR                 |
|-------------------------|-----------------------|
| Nombre de la red        | Hardhat local         |
| URL de RPC              | http://127.0.0.1:8545 |
| Identificador de cadena | 31337                 |
| Símbolo de moneda       | ETH                   |

Cámbiense a esa red. Su saldo será 0 ETH: es normal.

3A.3 · Terminal 2: desplieguen la FichaUSB y mándense fondos. DESTINO es SU dirección de MetaMask:

```
cd "ruta\a\laboratorios-evm"
$env:DESTINO="0xSU_DIRECCION_DE_METAMASK"
npx hardhat run scripts/s12/preparar-local.js --network localhost
```

(macOS/Linux: DESTINO=0x... npx hardhat run scripts/s12/preparar-local.js --network localhost.)

Salida esperada:

```
FichaUSB desplegada en la red local
-----
DIRECCION_FICHA : 0x5FbDB2315678afecb367f032d93F642f64180aa3
chainId         : 31337
a 0xSU_DIRECCION
  FUSB          : 1000.0
  ETH (local)   : 10.0
```

La dirección del contrato puede ser otra si ya desplegaron algo en ese nodo: copien **la que imprima**. En MetaMask ya deberían ver 10 ETH. Opcional: *Importar tokens* con la dirección del contrato para ver los 1000 FUSB en la billetera.

#### SI REINICIAN EL NODO

Todo se borra: contrato, saldos, historial. Hay que volver a correr `preparar-local.js`, pegar la nueva dirección en la dApp y, en MetaMask, borrar los datos de actividad de la cuenta (en configuración avanzada: *Borrar datos de la pestaña de actividad* o similar), o aparecerá el error «nonce too high».

## Opción B • Sepolia

Usen la dirección de su FichaUSB de la Sesión 10. Necesitan en MetaMask (red Sepolia) algo de ETH de prueba para el gas y algunos FUSB: si no tienen FUSB, quien desplegó el contrato los acuña con `acunar` desde el explorador (*Write Contract*). La red esperada será `11155111n`.



El ciclo de trabajo es siempre el mismo: **editar** → **guardar (Ctrl+S)** → **recargar el navegador (F5)** → **probar**. No se compila nada. Los comentarios de cada TODO traen el código casi literal: lo importante es entender cada línea.

**1**

TODO 1 · ESTADO DE ERROR 2

### Billetera no instalada

**Qué hace:** al cargar la página, si `window.ethereum` no existe, muestra el aviso y desactiva el botón Conectar.

**Qué escribir:** `if (typeof window.ethereum === "undefined") { $("sinWallet").hidden = false; $("btnConectar").disabled = true; }`

**Por qué `typeof`:** preguntar por una variable que no existe con `typeof` no lanza error; usarla directamente, sí.

**Cómo lo compruebo:** abran la misma URL en un navegador o una ventana sin MetaMask (por ejemplo Edge, o una ventana de invitado): texto rojo «No se detecta una billetera...» y botón gris. Esa es la **captura 1**.

**2**

TODO 2 · CONECTAR

### Provider, permiso, Signer y cuenta

**Qué escribir** (dentro del `try`, en lugar de la línea `return falta(...)`, que se borra):

`provider = new ethers.BrowserProvider(window.ethereum);` — el Provider habla con el nodo a través de MetaMask.

`await provider.send("eth_requestAccounts", []);` — aquí MetaMask pregunta si autorizan a este sitio a ver su cuenta.

`signer = await provider.getSigner();` y `cuenta = await signer.getAddress();`

y mostrar: `const red = await provider.getNetwork();`, la cuenta abreviada, `red.name + " (" + red.chainId + ")"`, y `$("infoConexion").hidden = false;`

**Cómo lo compruebo:** al pulsar Conectar, MetaMask pide permiso la primera vez; luego aparecen «Cuenta 0x1234...abcd» y «Red unknown (31337)» (en red local ethers no conoce un nombre para 31337) o «sepolia (11155111)».

**3**

TODO 3 · ESTADO DE ERROR 1

### Red equivocada

**Qué escribir:** si `red.chainId !== CHAIN_ID_SEPOLIA`: mostrar `$("avisoRed")` con un texto que diga a qué red cambiar, `$("btnEnviar").disabled = true`, `estado("Red equivocada: ...", "error")` y `return`. Si coincide, ocultar el aviso.

**Cómo lo compruebo:** lo harán en el paso 7. Si al conectar dice «red equivocada» estando en la red correcta, revisen la `n` de 31337n.



## El puente hacia la FichaUSB

**Qué escribir:** `contrato = new ethers.Contract(DIRECCION_FICHA, ABI, signer);`, `simbolo = await contrato.symbol();`, `decimales = Number(await contrato.decimals());` y luego `await refrescarSaldo();` `suscribirEventos();` `$("#btnEnviar").disabled = false;` `estado("Conectado. Listo para transferir.", "ok");`

**Por qué `Number()` aquí sí:** `decimals` es 18, un número pequeño. Con `MONTOS` nunca se usa `Number`: pierden precisión.

**No olviden `suscribirEventos()`**: en la dApp de referencia faltaba esa llamada y el saldo no se actualizaba solo. Fue un error real, encontrado al probarla.

`balanceOf`, gratis

**Qué escribir:** `const bruto = await contrato.balanceOf(cuenta);` y `$("#saldo").textContent = ethers.formatUnits(bruto, decimales) + " " + simbolo;` (y borrar la línea `falta(...)`).

**Qué pasa por debajo:** un `eth_call`: el nodo ejecuta la función en su copia y responde. No hay transacción, no hay firma, no hay gas para ustedes.

**Punto de control 3:** al conectar, «Saldo FUSB **1000.0 FUSB**» y la caja Estado en verde «Conectado. Listo para transferir.».

## 6

TODO 6 · ESCRITURA · transfer

Enviar y ESPERAR, con un mensaje en cada momento

**Qué escribir** (dentro del `try` del botón Enviar, en lugar de `falta(...)`):

```
const cantidad = ethers.parseUnits(monto, decimales); — «2.5» se vuelve
2500000000000000000n.
const tx = await contrato.transfer(dest, cantidad); — aquí MetaMask muestra la transacción y
pide confirmar.
estado("Transacción enviada. Esperando confirmación... " + tx.hash.slice(0, 10) + "...",
"pendiente");
const recibo = await tx.wait(); — aquí espera a que se MINE.
if (recibo.status === 0) { ... "se revirtió" ...; return; }, await refrescarSaldo(); y el
estado verde con recibo.blockNumber.
```

**El error que se combate:** decir «listo» después de `transfer()` y antes de `wait()`. El primer `await` solo significa «enviada».

**Punto de control 4:** transfieran 2,5 FUSB a la dirección de un compañero (o a una segunda cuenta suya). Se ve amarillo «Confirma la transacción en la billetera...», luego amarillo «Transacción enviada. Esperando confirmación... 0x...», y al final verde «¡Listo! Transferencia confirmada en el bloque N.»; el saldo baja a 997.5. En red local todo pasa en menos de un segundo; en Sepolia, unos 12 segundos.

## 7

TODO 7 · manejarError(e)

Un mensaje claro para cada caso

**Qué escribir**, en este orden, cada caso con `estado(..., "error")` y `return`:

- a) **Rechazo (estado 3):** `e.code === "ACTION_REJECTED" || e.code === 4001`. El 4001 es el código de EIP-1193 («el usuario rechazó»); ethers v6 lo traduce a `ACTION_REJECTED`.
- b) **Revertida (estado 4):** `e.code === "CALL_EXCEPTION" || /revert/i.test(e.message || "")`. Si existe `e.receipt`, la transacción se minó y el gas SÍ se pagó; si no existe, falló al simularla (`estimateGas`) y no se gastó nada. Díganlo en el mensaje.
- c) **Sin ETH para el gas:** `e.code === "INSUFFICIENT_FUNDS"`.
- d) **Cualquier otro:** `"Error: " + (e.reason || e.shortMessage || e.message)`.

**Dato verificado:** en ethers v6, cuando una transacción minada queda con `status 0`, `tx.wait()` no devuelve el recibo: lanza `CALL_EXCEPTION` con el recibo adjunto. Por eso la reversión se atrapa aquí y no en el `if (recibo.status === 0)`, que queda como red de seguridad.

## Que el saldo cambie solo

**Qué escribir:** `contrato.on("Transfer", (from, to) => { if (from === cuenta || to === cuenta) refrescarSaldo(); });`

**Cómo lo compruebo en red local:** con la dApp conectada, en la terminal 2 corran otra vez el script, ahora con la variable `FICHA` : no despliega nada, solo les acuña 100 FUSB más.

```
$env:FICHA="0x5FbDB..."; npx hardhat run scripts/s12/preparar-local.js --network localhost
```

**Salida:** Acuñados 100 FUSB más a 0x... · Saldo ahora: 1097.5 FUSB . Miren la dApp **sin recargar**: en unos segundos el saldo cambia solo. En la preparación, el saldo mostrado pasó de 1083,99... a 1183,99... en menos de 5 segundos.

**En Sepolia:** pídanle a un compañero que les transfiera desde su dApp.

## 09 Paso 7 · Provocar los cuatro errores a propósito

15 MIN · 6 CAPTURAS

Una dApp que solo maneja el camino feliz miente la mitad del tiempo. Aquí se demuestra que la suya no. Tomen una captura de pantalla de cada fila, con la caja «Estado» visible.

| ESTADO             | CÓMO PROVOCARLO                                                                                                                                                             | QUÉ DEBE MOSTRAR (MENSAJE REAL DE LA SOLUCIÓN)                                                                                                                 |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 · Sin billetera  | Abrir <code>http://localhost:3000</code> en un navegador o ventana sin MetaMask.                                                                                            | «No se detecta una billetera. Instala MetaMask y recarga esta página.» y el botón Conectar gris.                                                               |
| 1 · Red equivocada | En MetaMask, cambiar a otra red (p. ej. Sepolia si esperan 31337). La página se recarga sola (evento <code>chainChanged</code> ); pulsen Conectar otra vez.                 | «Estás en la red equivocada. Cambia a ... en la billetera y reconecta.» y el botón Enviar desactivado.                                                         |
| 3 · Rechazo        | Llenar destino y monto, Enviar, y en la ventana de MetaMask pulsar <b>Rechazar</b> .                                                                                        | «Rechazaste la transacción en la billetera.» y el botón Enviar vuelve a estar activo.                                                                          |
| 4 · Revertida      | Transferir más FUSB de los que tienen, p. ej. <code>5000000</code> .                                                                                                        | «La transacción se revirtió al simularla, antes de firmar (no se gastó gas). Motivo: execution reverted (unknown custom error)». MetaMask ni siquiera se abre. |
| Pendiente          | En Sepolia, capturar justo después de confirmar en MetaMask. En local es instantáneo: capturen el amarillo «Confirma la transacción en la billetera...» antes de confirmar. | Caja amarilla, con el hash cuando ya fue enviada.                                                                                                              |
| Confirmada         | Una transferencia normal.                                                                                                                                                   | «¡Listo! Transferencia confirmada en el bloque N.» en verde.                                                                                                   |

### POR QUÉ «UNKNOWN CUSTOM ERROR»

La FichaUSB revierte con un error personalizado de OpenZeppelin, `ERC20InsufficientBalance(sender, balance, needed)`. Como el ABI de la dApp no lo declara, ethers no sabe decodificarlo. **Reto opcional:** agreguen al ABI la línea `"error ERC20InsufficientBalance(address sender, uint256 balance, uint256 needed)"` y miren cómo cambia el motivo. Con MetaMask real el texto puede variar un poco según la versión; lo que se evalúa es que se clasifique como revertida.

### ¿Y UNA REVERTIDA YA MINADA?

Pasa cuando la simulación sale bien pero la ejecución real falla (el estado cambió entre medio), o cuando alguien fuerza el envío. En la red local, Hardhat rechaza incluso las transacciones forzadas que revertirían, así que ese caso se ve mejor en Sepolia. Su TODO 7 ya lo cubre: `CALL_EXCEPTION` con `e.receipt`.

## 10 Cómo saber que terminaron

---

- Ninguna acción muestra ya «Falta completar el TODO...».
- Conectan, ven su saldo, transfieren y el saldo baja; el mensaje final incluye el número de bloque.
- Tienen las seis capturas de la sección 9.
- El saldo cambió sin recargar cuando alguien más les transfirió o les acuñó.
- Si la guardaron en el repositorio, `git status` no muestra ningún archivo con claves.

Si quieren comparar con una solución, pídánla al docente después de entregar: la dApp de referencia completa existe y es la que se usó para verificar cada salida de esta guía.

## 11 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

| SÍNTOMA                                                                  | CAUSA Y SOLUCIÓN                                                                                                                                                     |
|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| «No se detecta una billetera»<br>teniendo MetaMask                       | Abrieron el archivo con doble clic ( <code>file://</code> ). Sírvanlo con <code>npx serve .</code> o <code>python -m http.server</code> (paso 2).                    |
| <code>ethers is not defined</code> en la consola                         | No cargó ethers desde jsDelivr: sin internet o bloqueado. Recarguen con conexión; la primera carga necesita internet, luego queda en caché.                          |
| <code>could not decode result data (value="0x")</code>                   | En <code>DIRECCION_FICHA</code> no hay contrato en ESA red: dirección mal copiada, red equivocada, o reiniciaron el nodo. Corran <code>comprobar-ficha.js</code> .   |
| Siempre dice «red equivocada»                                            | <code>CHAIN_ID_SEPOLIA</code> no coincide con la red de MetaMask, o le falta la <code>n</code> ( <code>BigInt</code> ). 31337n en local, 11155111n en Sepolia.       |
| MetaMask: «nonce too high» o transacciones atascadas en local            | Reiniciaron el nodo y MetaMask recuerda la cadena vieja. Borren los datos de actividad de la cuenta en la configuración avanzada de MetaMask.                        |
| MetaMask no se abre al pulsar Enviar                                     | Si la caja dice «revertida al simularla», es lo esperado (monto mayor al saldo). Si no dice nada, miren la consola: probablemente un error de sintaxis en su TODO 6. |
| Todo error sale como «sin clasificar todavía, TODO 7»                    | Aún no completaron <code>manejarError</code> , o el orden de los <code>if</code> está mal: el rechazo va antes que la reversión.                                     |
| El saldo no se actualiza solo                                            | Falta <code>suscribirEventos()</code> después de conectar (TODO 4) o el cuerpo del TODO 8.                                                                           |
| Saldo con muchos decimales raros o <code>1e+21</code>                    | Usaron <code>Number</code> o <code>toString</code> en vez de <code>ethers.formatUnits(bruto, decimales)</code> .                                                     |
| <code>invalid BigNumberish string: Cannot convert 2.5 to a BigInt</code> | Pasaron el texto del monto directo a <code>transfer</code> . Conviértanlo: <code>ethers.parseUnits(monto, decimales)</code> .                                        |
| <code>invalid FixedNumber string value</code>                            | El monto llegó con coma decimal («2,5»). Escribanlo con punto: <code>2.5</code> .                                                                                    |
| <code>npx serve</code> no se deja instalar                               | Usen <code>python -m http.server 8000</code> .                                                                                                                       |
| <code>insufficient funds</code> en red local                             | El script mandó el ETH a otra dirección: revisen <code>DESTINO</code> y que MetaMask esté en la cuenta correcta.                                                     |

## 12 Preguntas de reflexión

SE RESPONDEN MIRANDO SU PROPIA DAPP

Cada respuesta debe citar algo que ustedes vieron: un hash, un número de bloque, una línea de su código, un mensaje de su pantalla.

1. **Dos esperas.** Anoten, de una transferencia suya, el hash que se mostró en amarillo y el bloque que se mostró en verde. ¿Qué línea de su código produjo cada mensaje, y qué había pasado en la cadena en cada momento?
2. **Unidades.** En la salida de `comprobar-ficha.js`, ¿qué número exacto es el «saldo bruto»? ¿Qué número exacto viajó a la cadena cuando transfirieron 2,5 FUSB, y qué función lo produjo?
3. **Revertida sin gas.** Al provocar el estado 4, ¿se abrió MetaMask? ¿Se gastó ETH? Expliquen quién detectó la reversión y en qué momento (pista: `estimateGas` ).
4. **Qué NO está descentralizado.** Dibujen la cadena de piezas de SU dApp (servidor local, MetaMask, RPC, nodo, contrato). ¿Cuál de ellas, si cae, impide usarla, y cuál no afecta a los saldos? ¿Qué cambiarían para publicarla de verdad?
5. **Eventos.** ¿Cuánto tardó su dApp en reflejar la transferencia o el acuñado externo? ¿Qué pasaría con el saldo mostrado si quitan `suscribirEventos()` ?

## 13 Lista de chequeo de entrega

| ✓                        | ÍTEM                                                                                                                      |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | <code>index.html</code> con los 8 TODO completos, en el repositorio del equipo.                                           |
| <input type="checkbox"/> | Captura 1 · sin billetera.                                                                                                |
| <input type="checkbox"/> | Captura 2 · red equivocada.                                                                                               |
| <input type="checkbox"/> | Captura 3 · rechazo del usuario.                                                                                          |
| <input type="checkbox"/> | Captura 4 · transacción revertida.                                                                                        |
| <input type="checkbox"/> | Captura 5 · pendiente (amarillo).                                                                                         |
| <input type="checkbox"/> | Captura 6 · confirmada (verde, con bloque) y el hash de esa transacción (en Sepolia, enlace al explorador).               |
| <input type="checkbox"/> | Evidencia de actualización por evento (dos capturas: antes y después, sin recargar).                                      |
| <input type="checkbox"/> | Respuestas a las cinco preguntas de la sección 12.                                                                        |
| <input type="checkbox"/> | Ninguna clave privada, frase semilla ni URL de RPC con clave en el repositorio ( <code>git status</code> antes del push). |

## 14 Rúbrica

| CRITERIO      | PESO | EXCELENTE (5,0)                                                                               | ACEPTABLE (3,5)                                                     | INSUFICIENTE (<3,0)                    |
|---------------|------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------|
| Código        | 25 % | Los 8 TODO completos; montos siempre como BigInt; botón desactivado mientras se envía.        | Funciona, con algún atajo (p. ej. <code>Number</code> en un monto). | Faltan TODO o la dApp no conecta.      |
| Transferencia | 20 % | Confirmada, con hash verificable en Sepolia.                                                  | Confirmada solo en red local.                                       | Sin transferencia.                     |
| Estados       | 35 % | Seis capturas; cada error con su mensaje propio y el estado 4 distingue simulación de minada. | Cuatro o cinco estados bien manejados.                              | Menos de cuatro, o mensajes genéricos. |
| Eventos       | 10 % | El saldo cambia solo, con evidencia.                                                          | Funciona pero sin evidencia clara.                                  | No funciona.                           |
| Reflexión     | 10 % | Cinco respuestas con datos propios.                                                           | Correctas pero genéricas.                                           | Faltan o son incorrectas.              |



## 15 Referencias

---

- ethers v6, documentación: [docs.ethers.org/v6/](https://docs.ethers.org/v6/)
- EIP-1193, *Ethereum Provider JavaScript API* (códigos de error como 4001). [eips.ethereum.org/EIPS/eip-1193](https://eips.ethereum.org/EIPS/eip-1193)
- EIP-6963, *Multi Injected Provider Discovery* (Final). [eips.ethereum.org/EIPS/eip-6963](https://eips.ethereum.org/EIPS/eip-6963)
- MetaMask, documentación para desarrolladores. [docs.metamask.io](https://docs.metamask.io)
- Hardhat 3, *hardhat node* y redes. [hardhat.org/docs](https://hardhat.org/docs)
- wagmi y RainbowKit (para el proyecto). [wagmi.sh](https://wagmi.sh) · [rainbowkit.com](https://rainbowkit.com)